OSSP (Operating Systems And Systems Programming) - 25CS2104E

WEEK-6:

Task 1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
aruntej@LAPTOP-DGRT8C5G:~$ cd ~
aruntej@LAPTOP-DGRT8C5G:~$ mkdir -p week6
aruntej@LAPTOP-DGRT8C5G:~$ cd week6
aruntej@LAPTOP-DGRT8C5G:~/week6$ nano task1.c
aruntej@LAPTOP-DGRT8C5G:~/week6$ gcc -Wall -Wextra task1.c -o task1
./task1
Task 1: Command Parser
Enter a command (type exit to quit)
parser> hello world
Parser output: hello world
Parser validation: PASS
parser> echo "hello world"
Parser output: echo hello world
Parser validation: PASS
parser> hello\;world
Parser output: hello;world
Parser validation: PASS
parser> echo "hello
Parser validation: FAIL
Reason: unmatched quote, trailing escape, or input too long
parser>exit
```

c code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1024

int parse_command(const char *input, char *output) {
    int i = 0, j = 0;
    char quote = '\0';

    while (input[i] != '\0' && input[i] != '\n') {
        char c = input[i];

        // Handle escaped characters, including spaces
        if (c == '\\') {
            if (input[i + 1] == '\0' || input[i + 1] == '\n')
                return -1;

            output[j++] = input[i + 1];
            i += 2;
        }
```

```
        // Handle single and double quotes
        else if (c == '\'' || c == '"') {
            if (quote == '\0')
                quote = c;
            else if (quote == c)
                quote = '\0';
            else
                output[j++] = c;

            i++;
        }
        // Preserve characters inside quotes
        else {
            if (quote == '\0' && c == ';') {
                printf("Special symbol detected: ;\n");
            }

            output[j++] = c;
            i++;
        }

        if (j >= MAX - 1)
            return -1;
    }

    if (quote != '\0')
        return -1;

    output[j] = '\0';
    return 0;
}

int main(void) {
    char input[MAX], output[MAX];

    printf("Task 1: Command Parser\n");
    printf("Enter a command (type exit to quit)\n");

    while (1) {
        printf("parser> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (parse_command(input, output) == 0) {
            printf("Parser output: %s\n", output);
            printf("Parser validation: PASS\n");
```

```
            } else {
                printf("Parser validation: FAIL\n");
                printf("Reason: unmatched quote, trailing escape, or input too long\n");
            }
        }

        return 0;
}
```

Task 2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```
aruntej@LAPTOP-DGRT8C5G:~/Week6$ nano task2.c
aruntej@LAPTOP-DGRT8C5G:~/Week6$ gcc -Wall -Wextra task2.c -o task2
./task2
Task 2: Process Launcher
Enter commands (type exit to quit)
launcher> pwd
ls -l
date
echo Hello World
invalidcommand123
exitParent PID: 693, Child PID: 694
/home/aruntej/Week6
Child exited with status: 0
launcher> Parent PID: 693, Child PID: 695
total 24
-rwxr-xr-x 1 aruntej aruntej 16600 Oct 13 03:23 task2
-rw-r--r-- 1 aruntej aruntej  1969 Oct 13 03:23 task2.c
Child exited with status: 0
launcher> Parent PID: 693, Child PID: 696
Tue Oct 13 03:23:50 UTC 2026
Child exited with status: 0
launcher> Parent PID: 693, Child PID: 697
Hello World
Child exited with status: 0
```

launcher> Parent PID: 693, Child PID: 698

Execution failed: No such file or directory

Child exited with status: 127

launcher> exit

Parent PID: 693, Child PID: 735

Execution failed: No such file or directory

Child exited with status: 127

C code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX 1024
#define MAX_ARGS 64

int main(void) {
    char input[MAX];

    printf("Task 2: Process Launcher\n");
    printf("Enter commands (type exit to quit)\n");

    while (1) {
        printf("launcher> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
```

```
input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;

if (input[0] == '\0')
    continue;

char *args[MAX_ARGS];
int count = 0;
char *token = strtok(input, " \t");

while (token != NULL && count < MAX_ARGS - 1) {
    args[count++] = token;
    token = strtok(NULL, " \t");
}

args[count] = NULL;

if (count == 0)
    continue;

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    continue;
}

if (pid == 0) {
```

```
            // Child process executes the requested program
            execvp(args[0], args);

            // Reached only if execvp fails
            perror("Execution failed");
            _exit(127);
        } else {
            // Parent waits for the child to finish
            int status;

            printf("Parent PID: %d, Child PID: %d\n",
                   (int)getpid(), (int)pid);

            if (waitpid(pid, &status, 0) == -1) {
                perror("waitpid failed");
                continue;
            }

            if (WIFEXITED(status)) {
                printf("Child exited with status: %d\n",
                       WEXITSTATUS(status));
            } else if (WIFSIGNALED(status)) {
                printf("Child terminated by signal: %d\n",
                       WTERMSIG(status));
            }
        }
    }

    printf("Parent process exiting.\n");
    return 0;
}
```